CATALOG

Every parliament fact, stored with its proof

Hack Apertus Online Hackathon · Track 2A, OpenParlData · Model: Apertus v1.5 8B · Harsh Panchal

1. Problem

Swiss parliaments publish debates as PDFs, and each of the 78 parliaments (federal, cantonal, municipal) uses its own layout, language and vocabulary. Turning those PDFs into structured data with an LLM is fast, but the result cannot be trusted by default: a model can paraphrase a speech, attach it to the wrong speaker or invent text that was never said. For a transparency project that is the worst failure, because a wrong record looks exactly like a right one and nobody can tell them apart.

2. Idea: a catalog where every fact carries its proof

CATALOG treats extraction as a bookkeeping problem. Every extracted fact becomes one entry in an **evidence catalog**. The entry holds the speaker, party, quote, source file, page number, the source text it came from, the matched topic keywords, and a status set by a validator that checks the quote against the page: **verified**, **flagged** or **rejected**. Only verified entries are exported to the OpenParlData schema. Flagged entries stay visible for human review, and rejected entries are kept as a record of what the model got wrong. The model is never trusted: it only proposes, and the validator decides.

3. Architecture

Component	File	Role
Layout catalog	src/catalogs.py	Regex patterns for known speaker-line layouts. Used as the fallback when no LLM is configured, and extendable per parliament.
Term catalog	src/catalogs.py	Normalises party names and vocabulary (SVP, Die Mitte, Grüne and so on).
Keyword catalog	src/keyword_catalog.py	Multilingual (DE/FR/IT) topic keywords. Each tag lists the exact keywords matched, so a topic can be traced to the quote.
Apertus prompt	src/llm.py	Asks Apertus for a JSON list of speaker, party and quote, with the quote copied verbatim. Temperature 0, OpenAI-compatible endpoint.
Validator	src/validator.py	Checks the quote against the page text and sets the status.
Evidence catalog	data/output/evidence_catalog.jsonl	All entries with page, source text, status and reason.
Export	data/output/openparldata_export.json	Verified entries only.

Validator rules. Text is compared after whitespace is normalised. A quote that appears on the page as a whole-word sequence and whose speaker is also on the page is **verified**. A quote that appears but with the speaker missing, or that matches the page at 85% or more of its length without being an exact whole-word match, is **flagged**. Anything else, including an empty quote, is **rejected**. The whole-word check means a truncated quote such as one ending in half a word cannot be verified.

Data flow. PDF pages are read with PyMuPDF, and each page goes to Apertus (or to the layout catalog when no model is configured). Every proposed entry is validated against the same page text, tagged by the keyword catalog and written to the evidence catalog. The export is derived from the evidence catalog

and never the other way around.

4. Evaluation

4.1 Validator self-test. The validator was run on 8 controlled cases built from a known page, covering exact quotes, formatting noise and the typical model errors. The outcome matched the expected status in **8 of 8** cases (verified 3, flagged 2, rejected 3). These are real outputs of `src/validator.py`, but they test the checking logic on a small constructed page and are **not** a measurement of extraction quality on real parliament PDFs.

Case	Status	Validator reason
Exact quote	verified	verbatim quote and speaker found on page
Exact quote, 2nd speaker	verified	verbatim quote and speaker found on page
Whitespace / line-wrap difference	verified	verbatim quote and speaker found on page
Truncated last word	flagged	near match (1.00)
Wrong speaker	flagged	quote found but speaker not on page
Paraphrase	rejected	quote not in source (0.33)
Invented sentence	rejected	quote not in source (0.18)
Empty quote	rejected	empty quote

4.2 Run on parliament PDFs.

Not yet measured. No `data/output/summary.json` was found when this report was built. Run `make run` on the parliament PDFs and rebuild this report with `python src/make_report.py`; the table is filled automatically from the run.

5. Limitations

• **Verification is textual, not semantic.** The validator proves a quote exists on the page; it cannot prove the model understood its meaning or attributed it to the right person when the same name appears twice on one page.
• **Layout catalog is hand-written.** Regex patterns cover common speaker-line formats and need extending for each parliament; the LLM path is more general but only as good as the prompt.
• **Scanned PDFs are not supported.** They need OCR first, and OCR errors would turn correct quotes into flagged ones.
• **Keyword catalog is a starter set.** Six topics in three languages; keyword matching ignores context and negation.
• **Export schema is a simplified mapping** of OpenParlData and should be checked against the official schema.
• **One validator threshold** (85%) was set by judgement and has not been tuned on real data.

6. How to run

Requires Docker. Put PDFs in `data/input/` (a sample PDF is generated if the folder is empty), then:

```
export LLM_NAME=<apertus-8b-v1.5 model name>
export LLM_BASE_URL=<OpenAI-compatible endpoint, e.g. https://host/v1>
export LLM_API_KEY=<key>
make run
```

Without the three variables the pipeline runs in layout-catalog mode. Results appear in `data/output/`: `evidence_catalog.jsonl`, `openparldata_export.json` and `summary.json`. A browser front end (`catalog.html`) loads the evidence catalog so every entry can be filtered by status or topic and opened to show its proof.